```
traces:
  receivers: [ otlp ]
  processors: [ batch ]
  exporters: [ jaeger ]
```

Receives spans from the instrumented ho11y services in OTLP format

Sends spans in batches rather than all at once

Ingests spans into the Jaeger backend

Note that, compared to the setup in chapter 6, I have removed the debug-level logging config for the collector here, since we don't need it and it makes it more distracting to follow along.

8.2.3 The "happy path"

Now, it's time to head over to http://localhost:16686/ to open the Jaeger UI, where we will first have a look at the "happy path" (see figure 8.5), meaning everything works as expected, and no errors are present. This gives us the baseline for what we would expect, in terms of how services should work (e.g., execution time duration).

As the initial step to explore and understand our distributed system, let's create the service map showing the (dynamically generated) overall app architecture. For that, select `frontend` in the left-hand-side box labeled Search, and then click the Find Traces button at the bottom. Now, you should see a graph of execution durations over time on the right-hand side, and if you now click the Deep Dependency Graph button right below it, you should see something akin to figure 8.5. Also, note what we don't see: the external services or calls to services are not represented as spans.

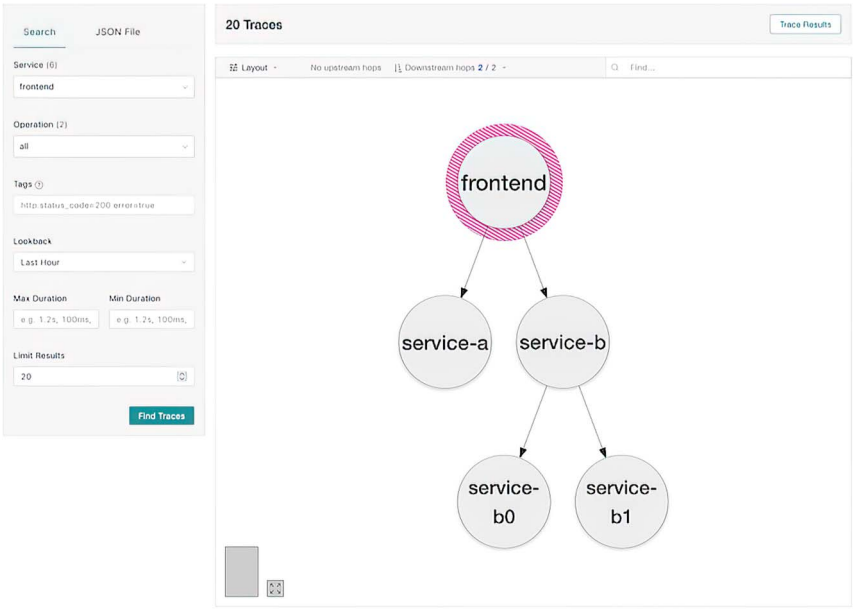


Figure 8.5 Troubleshooting demo microservices app: the happy path dependency graph